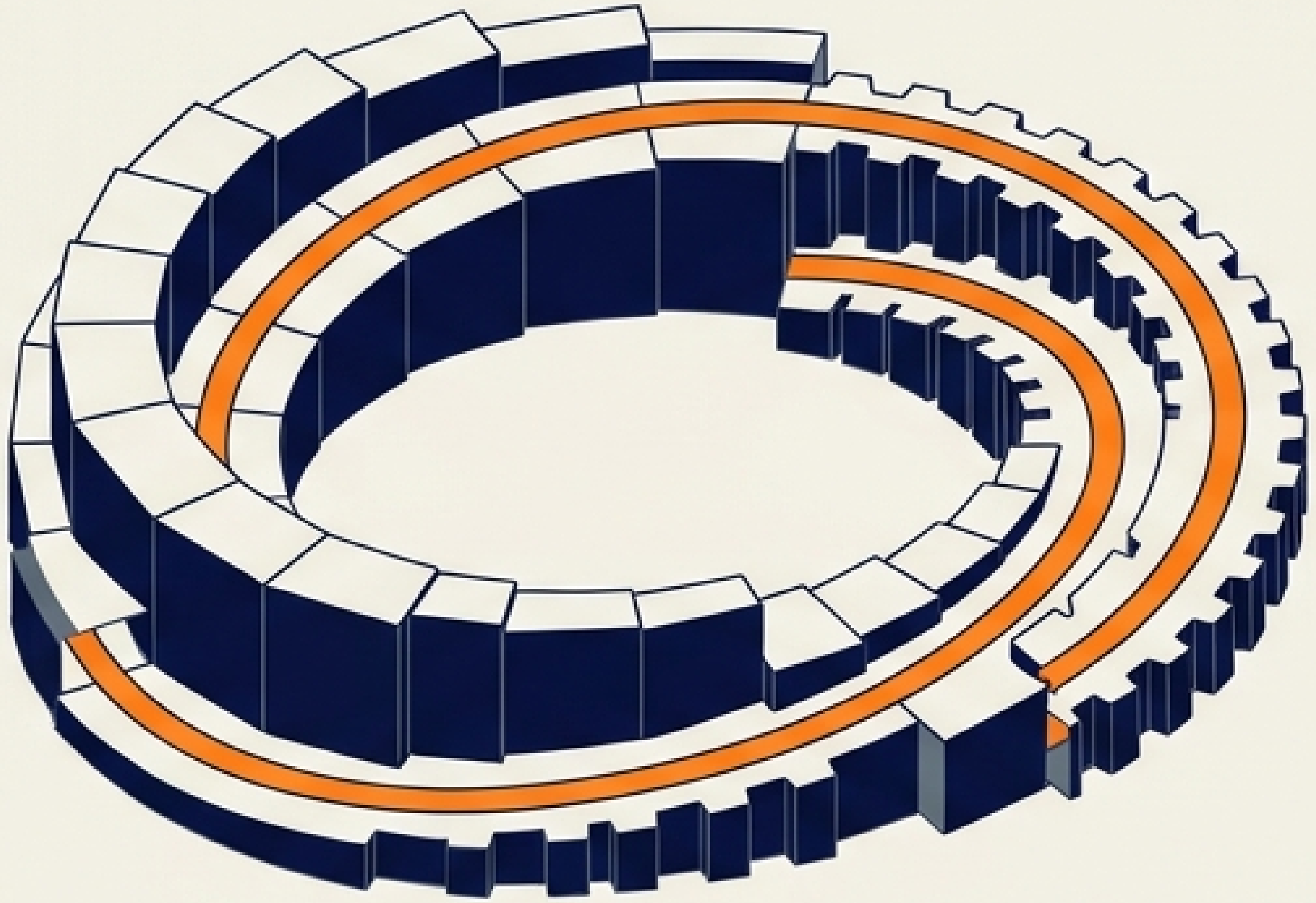
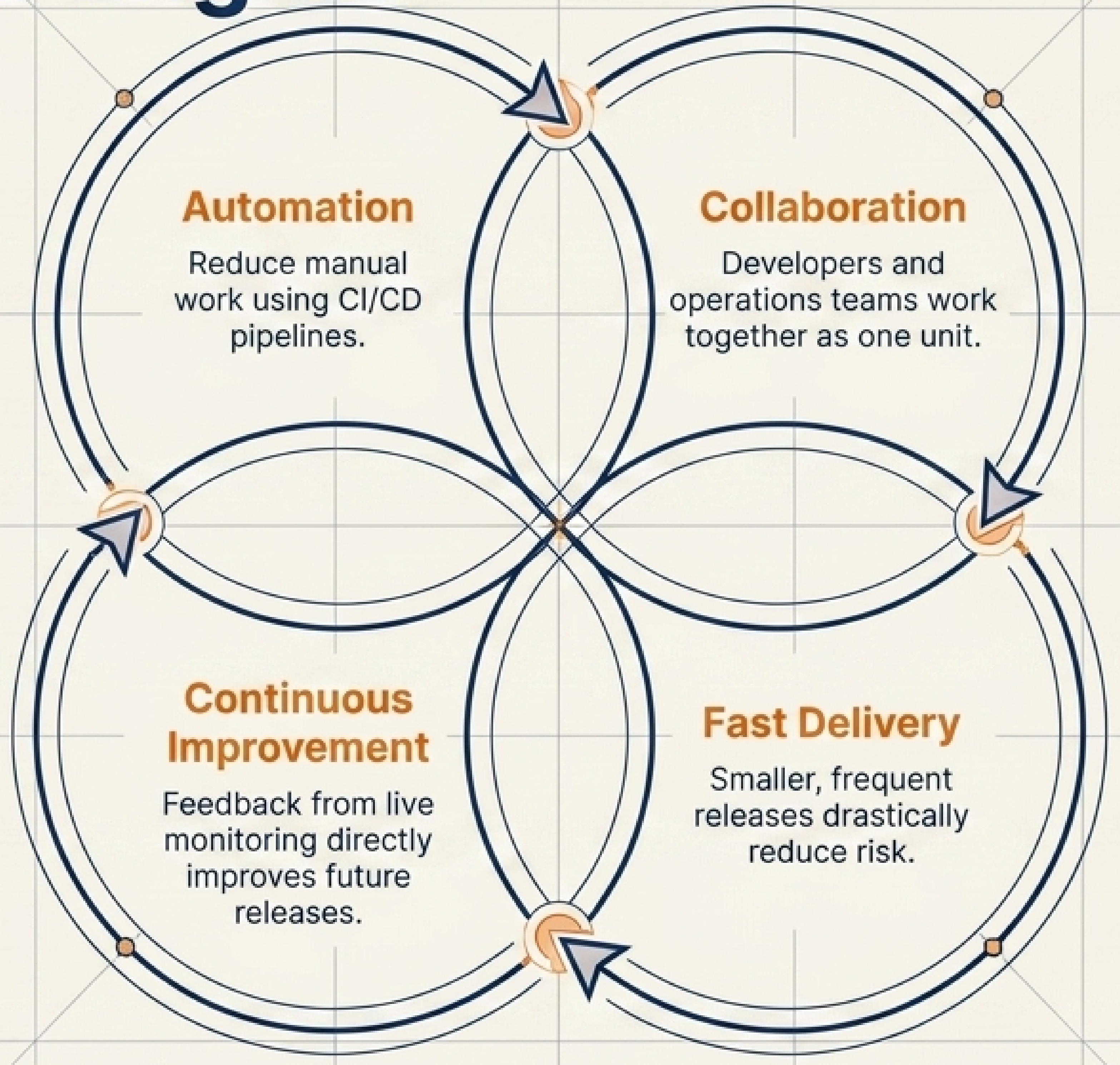


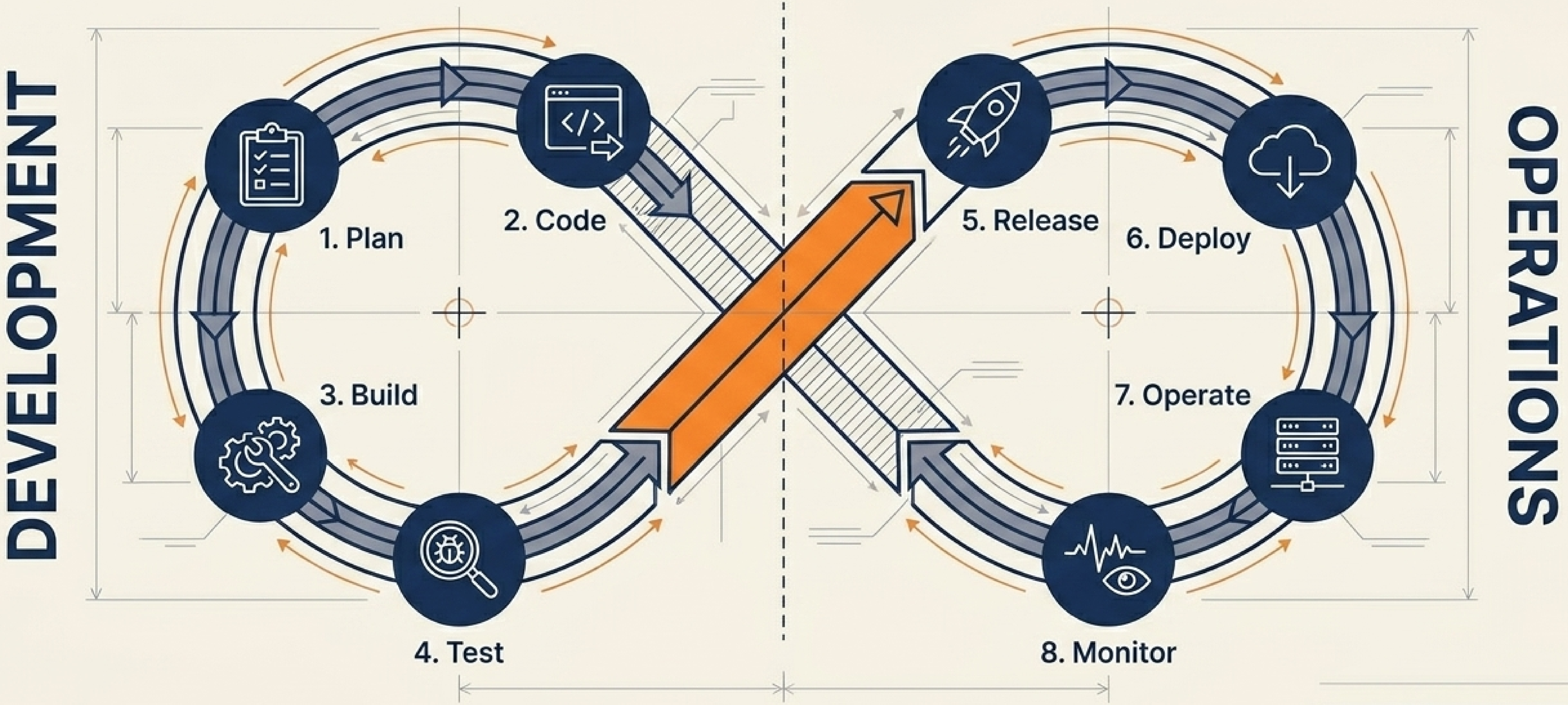
The 8 Pillars of the Continuous DevOps Lifecycle

Building, shipping, and improving software at the speed of modern engineering.

The Core Philosophy Operating the Engine



The Continuous Software Delivery Lifecycle



1. Plan: Defining the blueprint

Goals

- Gather requirements
- Define features & priorities
- Break work into tasks
- Estimate timelines
- Coordinate teams

Common Tools

- Jira
- Trello
- Azure DevOps
- Notion

Activities

Sprint planning → User stories → Backlog management → Architecture discussions → Risk analysis

Example: E-Commerce Wishlist

1	**Request:**	Users save products.
2	**PMs:**	Define requirements.
3	**Devs:**	Estimate effort.
4	**Assign:**	Frontend, backend, QA tasks.

2. Code: Writing and managing application source

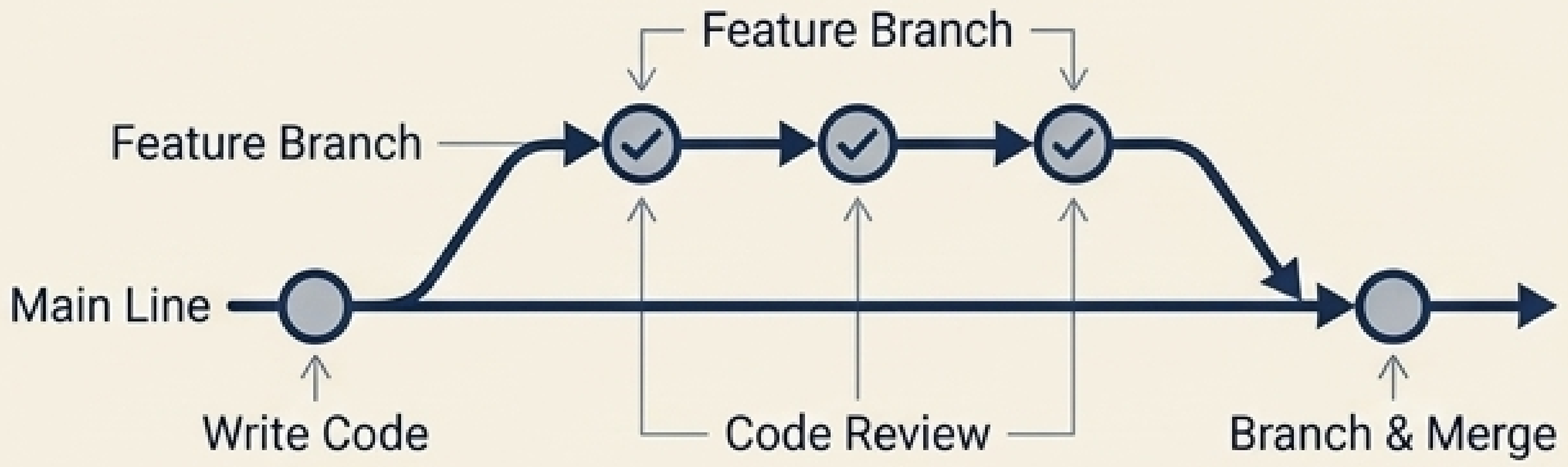
Goals

- Develop features
- Fix bugs
- Maintain code quality
- Collaborate safely

Version Control

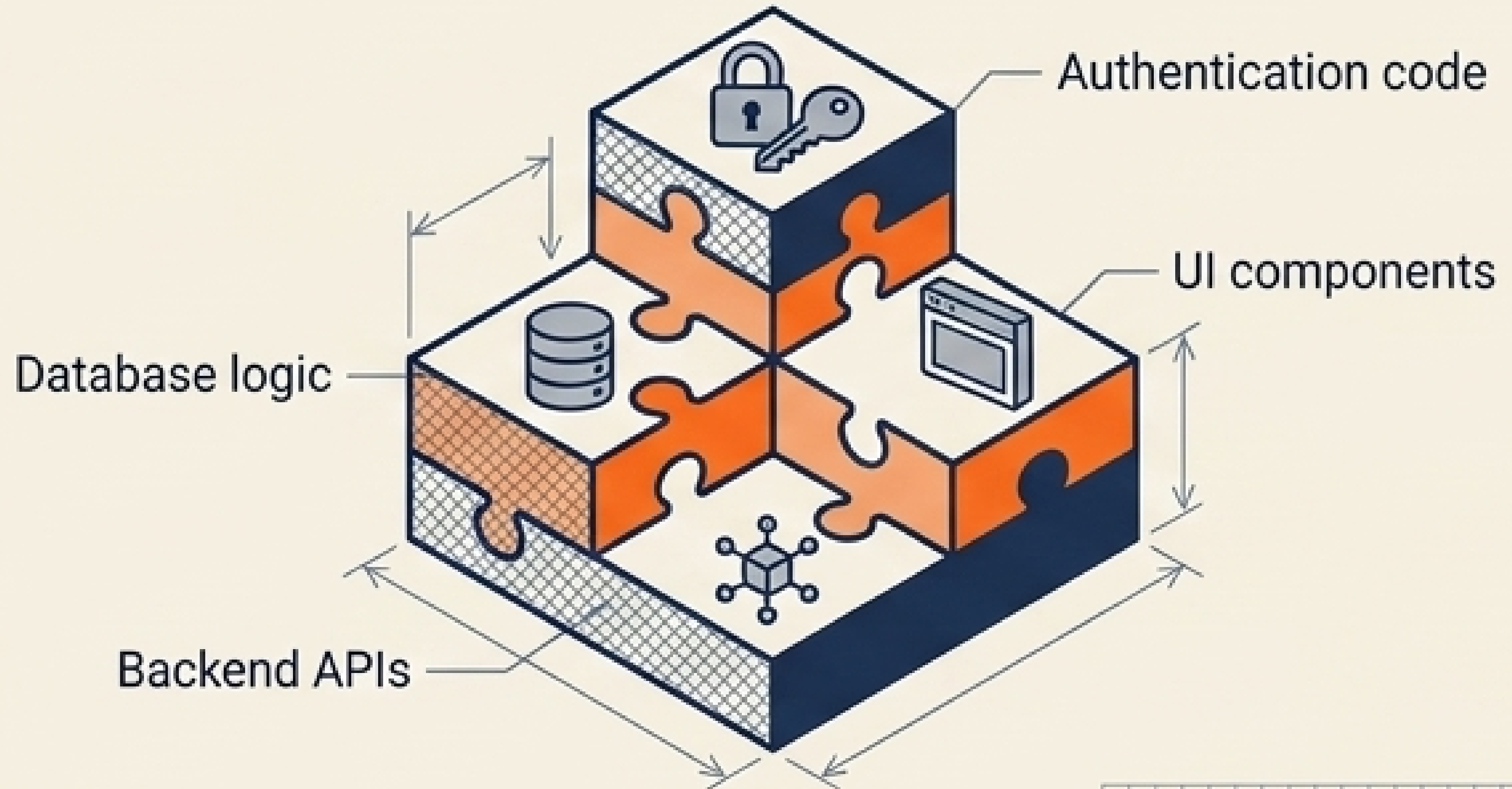
git · GitHub · GitLab · Bitbucket

Process Flow



Important Practice: Commit code frequently to a shared repository.

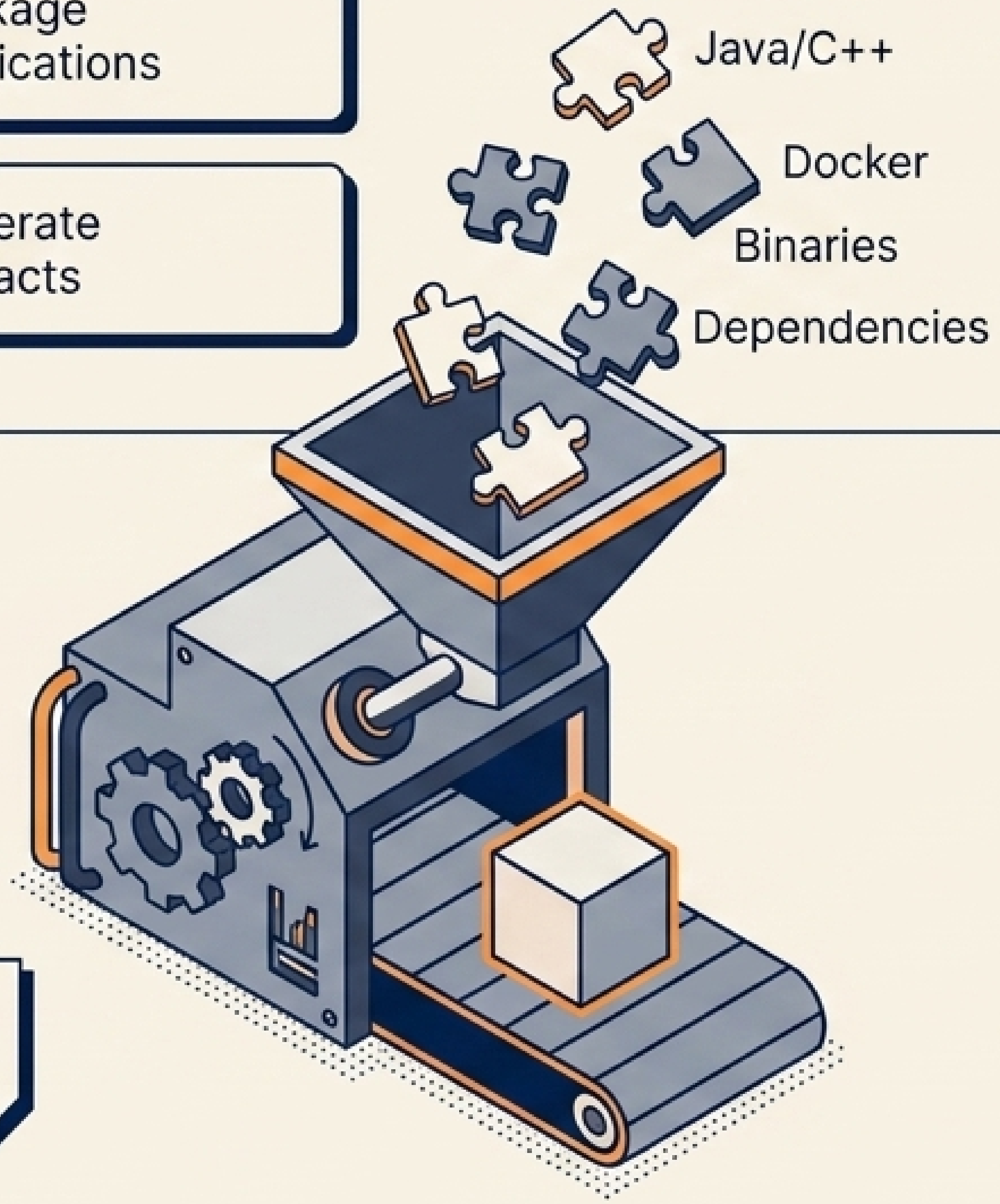
Example: Wishlist Components



3. Build: Converting code to a runnable application

Goals

- Compile code
- Package applications
- Resolve dependencies
- Generate artifacts



Build Tools

- Maven
- Gradle
- Docker
- Webpack

Automated Activities

CI Connection: This step is automatically triggered via Continuous Integration.

Example: Node.js App

Installs npm packages

↓

Creates production bundle

↓

Builds Docker container image

4. Test: Ensuring functionality and preventing regressions

Goals

- Detect bugs early
- Validate features
- Improve reliability
- Prevent regressions

Automation Frameworks

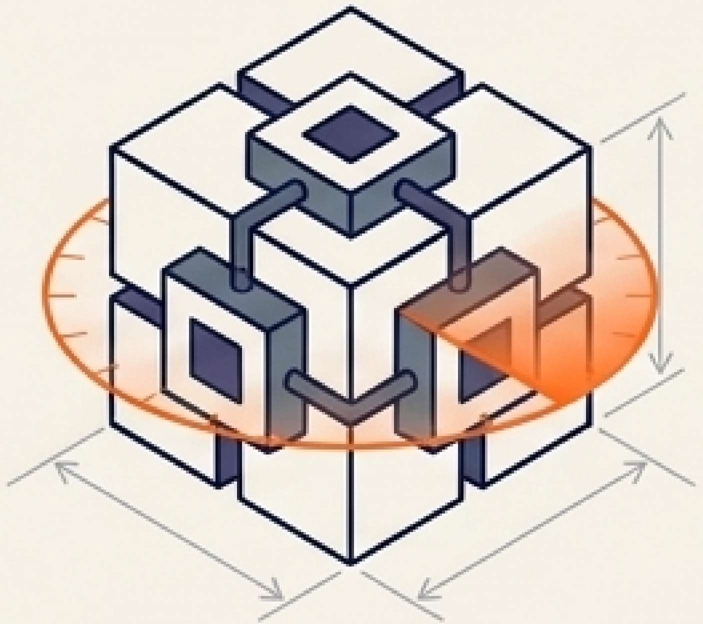
JUnit	Selenium
POSTMAN	cypress

Test Coverage

- Unit
- Integration
- API
- UI
- Security
- Performance

DevOps Principle: Testing must be automated.

Example: Wishlist Verification



- APIs are tested automatically
- Login flow is validated
- Security scans check vulnerabilities

5. Release: Preparing software for production delivery

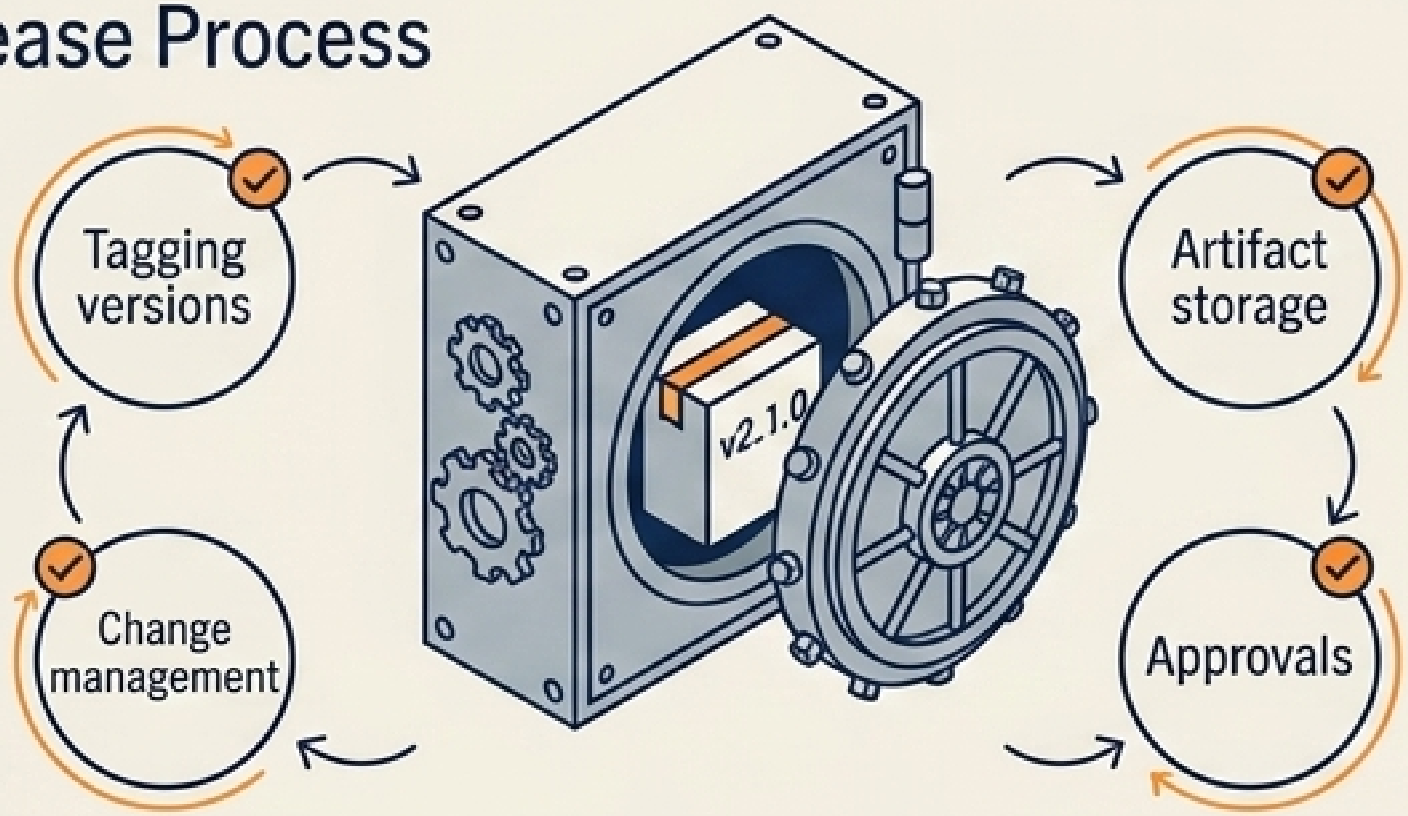
Goals

- Approve stable builds
- Manage versions
- Prepare release notes
- Coordinate deployment

Artifact Repositories

- Nexus Repository
- JFrog Artifactory
- GitHub Releases

Release Process



Example: Version v2.1.0 Approval

- All tests pass
- QA approval
- Security validation
- Documentation updates

6. Deploy: Pushing applications into live environments

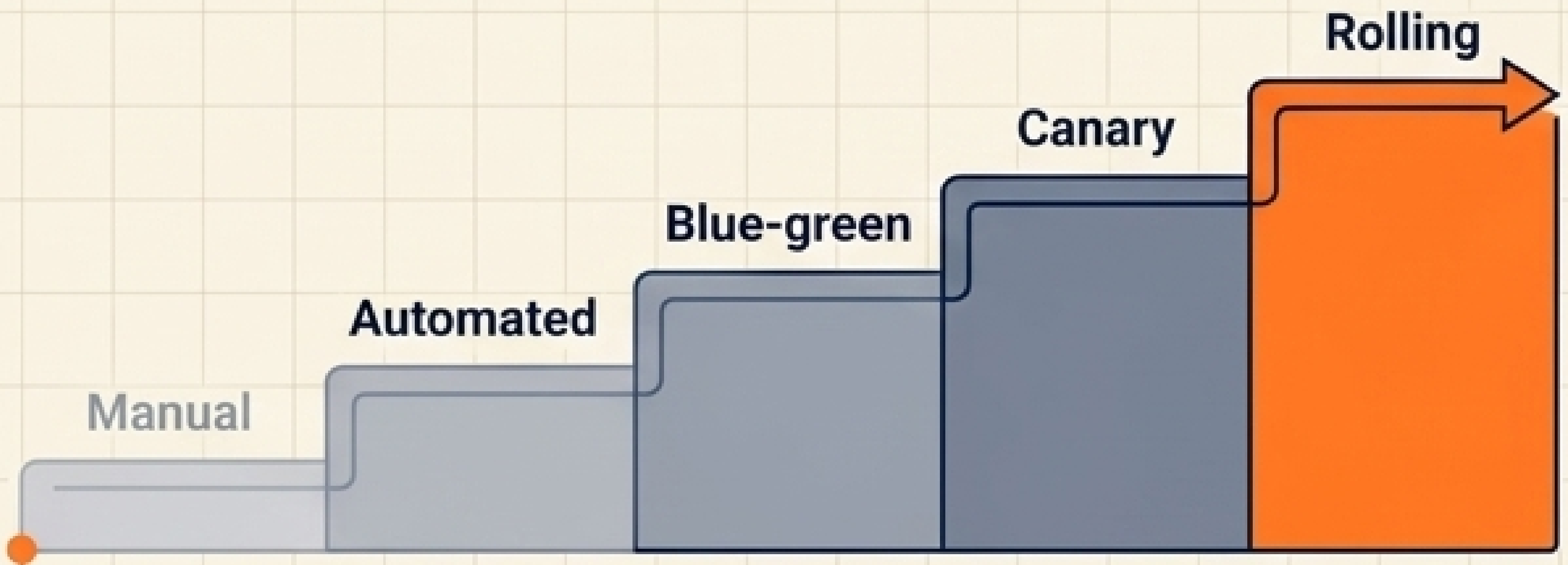
Goals

- Deliver software safely
- Reduce downtime
- Automate infrastructure
- Enable fast rollbacks

Orchestration Tools

- Kubernetes
- Argo CD
- Jenkins
- Ansible

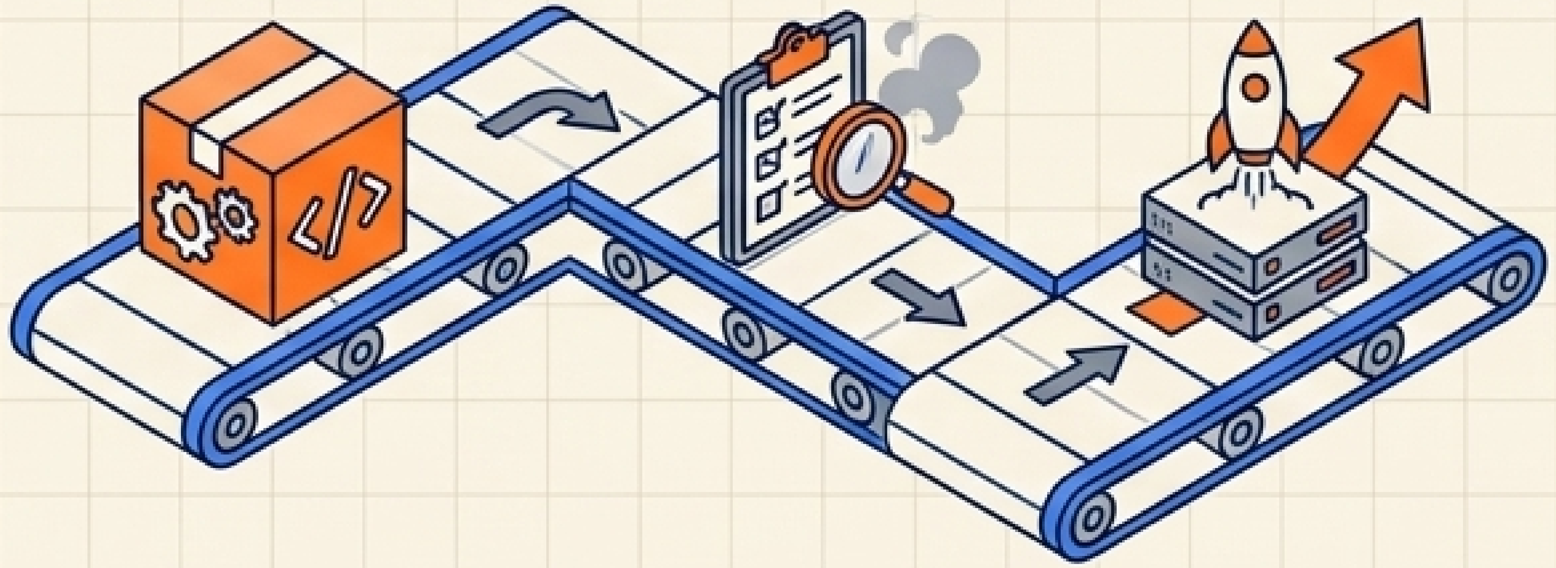
Deployment Maturity Strategy



DevOps Focus: Deployments should be repeatable and automated.

Example: Continuous Deployment Pipeline

- Deploy to staging
- Run smoke tests
- Deploy to production automatically



7. Operate: Maintaining the live production system

Goals

- Keep services running
- Manage infrastructure
- Handle incidents
- Scale systems

Cloud & Infrastructure Tools

- Kubernetes
- Terraform
- aws
- Azure

Operational Activities

- Server management
- Kubernetes cluster ops
- Backup handling
- Incident response
- Capacity scaling

Key Concept:
Infrastructure as Code (IaC) is heavily used here.

Example: Ops Team Response

Trigger: Traffic spikes during dinner rush.

Action	Result
• **Action 1:** Monitors server load.	☑
• **Action 2:** Adds new containers automatically.	☑
• **Action 3:** Handles potential outages.	☑

8. Monitor: Tracking system health and user experience

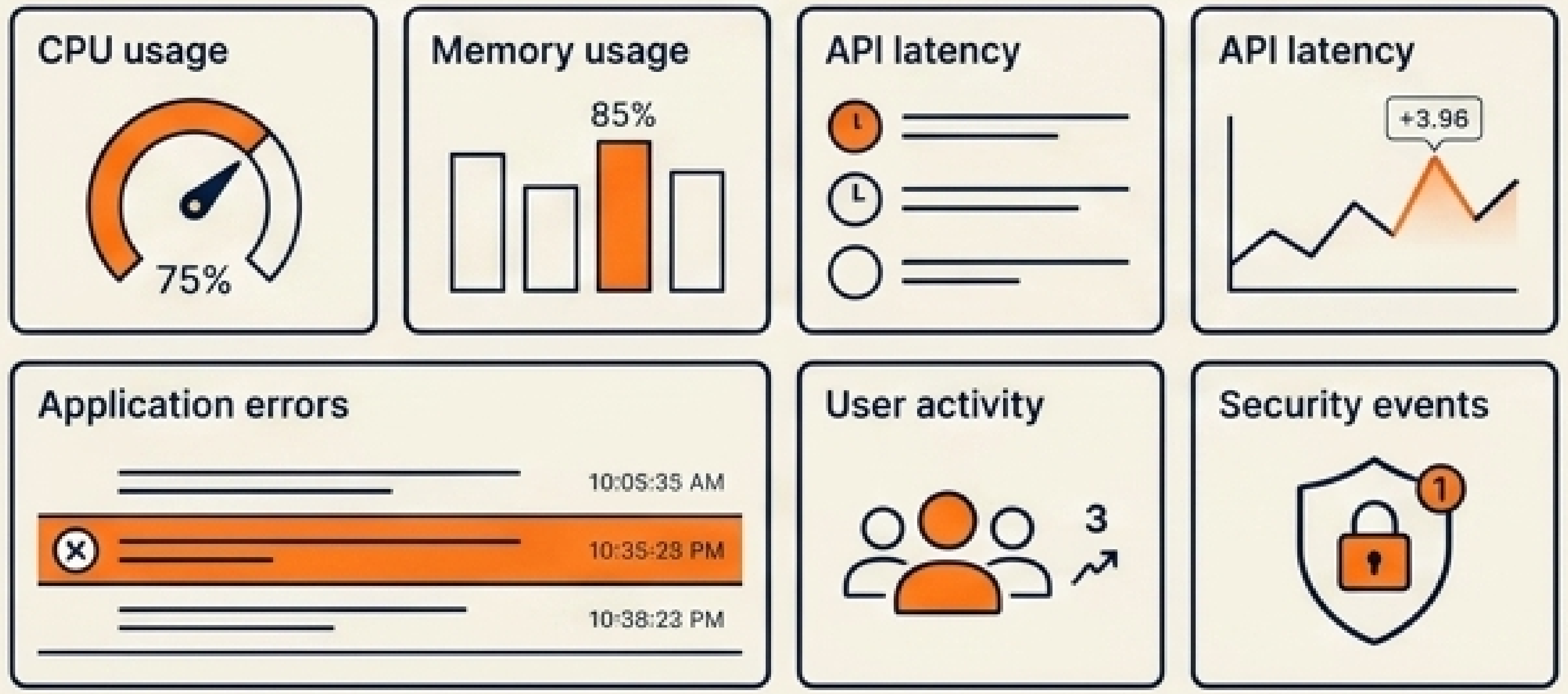
Goals

- Detect issues quickly
- Measure performance
- Improve reliability
- Gather business insights

Observability Tools

- Prometheus
- Grafana
- DATADOG
- New Relic

Telemetry Dashboard

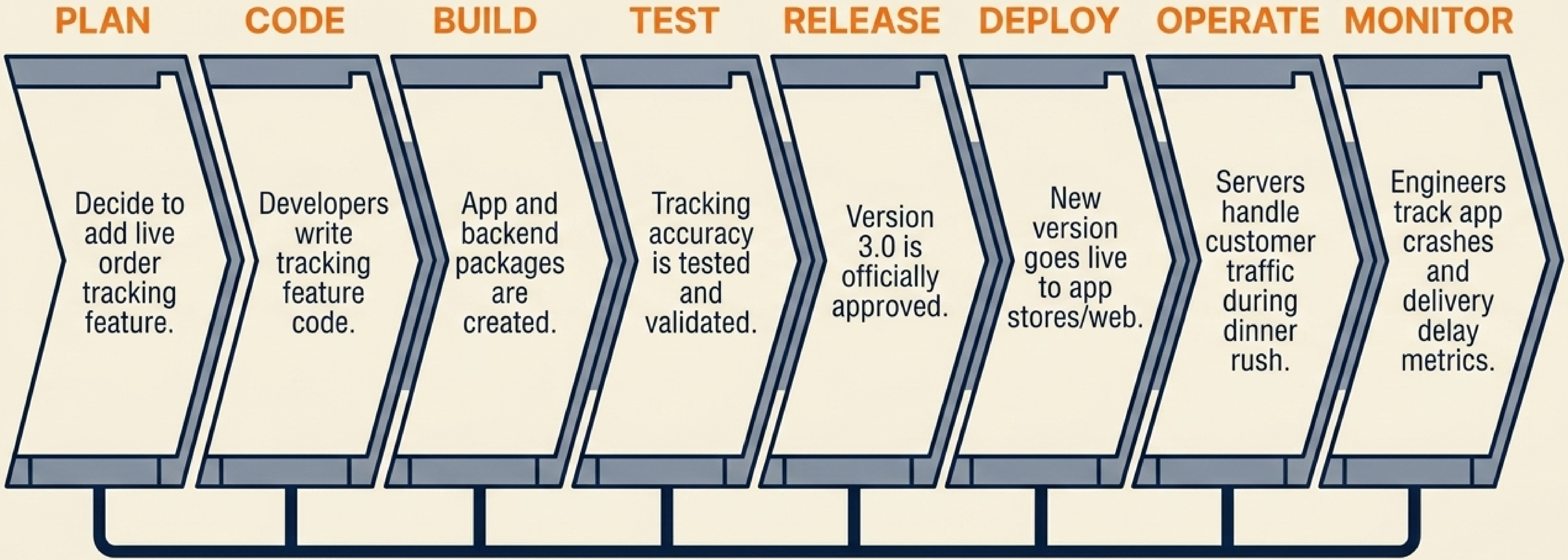


Example: Anomaly Detection

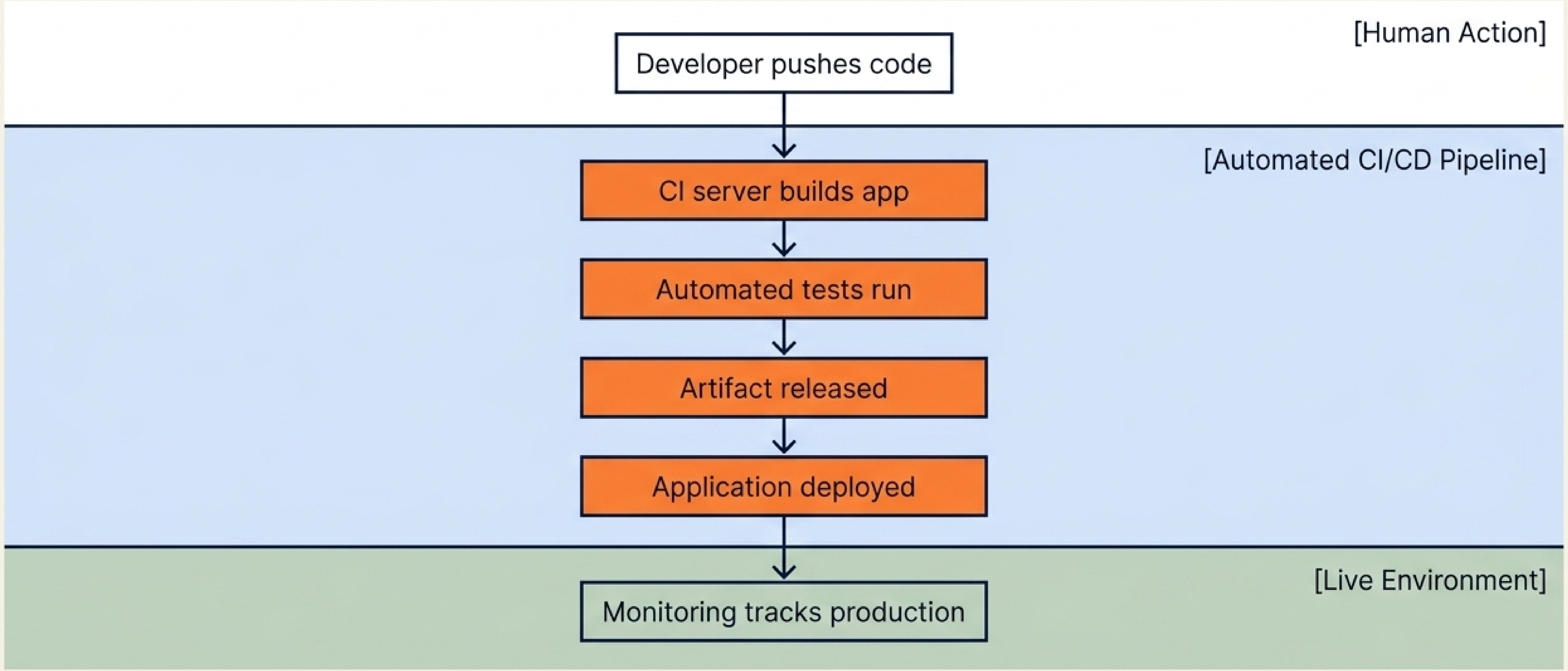
- High response time detected.
- Database slowdown.
- Failed login spikes.

Action: Alerts automatically sent to engineers.

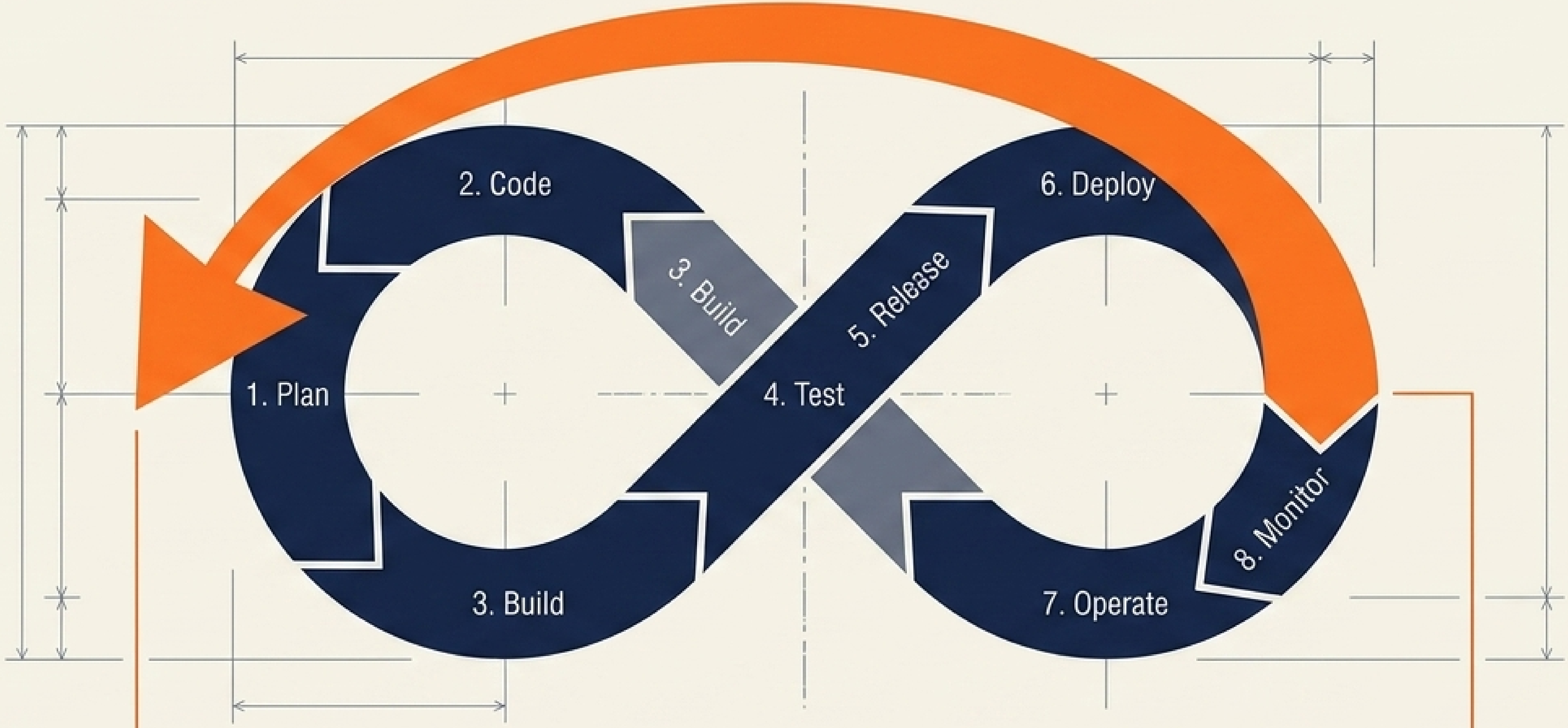
The Lifecycle in Action: Building a Food Delivery App



The Automation Waterfall: A Popular CI/CD Pipeline



The Continuous Feedback Loop



Monitoring feedback directly improves future planning and development.

The cycle never truly ends; it only accelerates and improves.